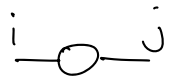
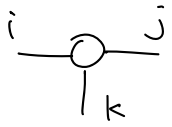
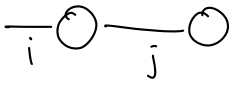
# 1. Introduction

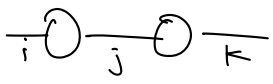
vector $v_j$


matrix $M_{ij}$ 

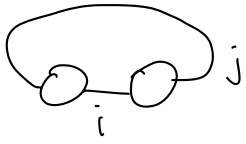

3-index tensor $T_{ijk}$ 


Operations: 

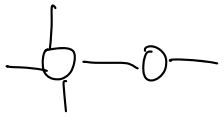
$= \sum_j M_{ij} v_j$

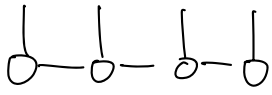


$= \sum_j M_{ij} N_{jk} = MN$



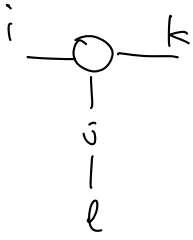
$= \sum_{i,j} M_{ij} N_{ji} = \mathrm{Tr}[MN]$


$= \sum_k T_{ijkl} V_{km}$


$= \sum_{\alpha_1 \alpha_2 \alpha_3} A^{s_1}_{\alpha_1} B^{s_2}_{\alpha_1 \alpha_2} C^{s_3}_{\alpha_2 \alpha_3} D^{s_4}_{\alpha_3}$

Identity matrices

$= \delta_{ij}$

$\sum_j T_{ijk} \delta_{jl} =$ 

$=$

$= T_{ilk}$

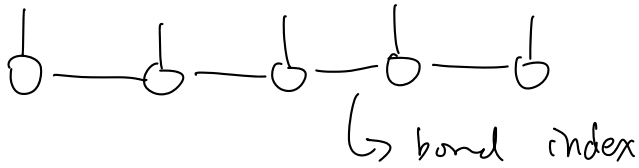
# 2. Matrix Product State / Tensor train

The MPS is a factorization of a tensor with $N$ indices into a chain-like product of three-index tensors.

$$T^{s_1 s_2 s_3 s_4 s_5} = \sum_{\{\alpha\}} A^{s_1}_{\alpha_1} B^{s_2}_{\alpha_1\alpha_2} C^{s_3}_{\alpha_2\alpha_3} D^{s_4}_{\alpha_3\alpha_4} E^{s_5}_{\alpha_4}$$

- <u>Bond dimension / rank</u>

The dimension of the bond index connecting one tensor in the chain to the next, and can vary from bond to bond.




- <u>Number of Parameters</u>

A tensor with $N$ indices, each of dimension $d$. $\Longrightarrow$ $d^N$ parameters.

Represent it by MPS of bond dimension $m$ requires $N d m^2$ parameters.

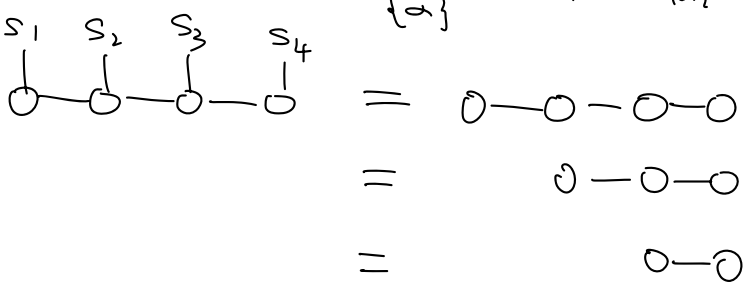
- <u>Elementary operations</u>

① Retrieving a component from an MPS

We want to obtain $T^{s_1 \cdots s_N}$, where $s_1, \cdots, s_N$ are fixed.

If $T$ is given by the MPS

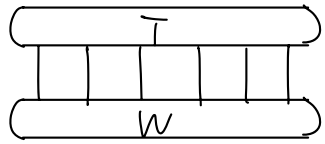
$$T^{s_1 \cdots s_N} = \sum_{\{\alpha\}} A^{s_1}_{\alpha_1} A^{s_2}_{\alpha_1\alpha_2} \cdots A^{s_N}_{\alpha_{N-1}}$$



## ② Inner Product of two MPS

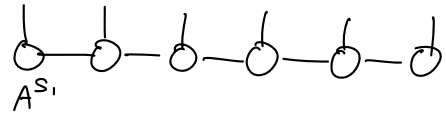
$T^{s_1 s_2 s_3 s_4 s_5 s_6}$ , $W^{s_1 s_2 s_3 s_4 s_5 s_6}$

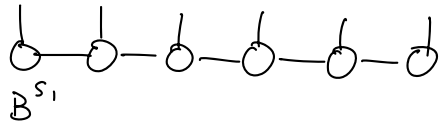
$$\langle T, W \rangle = \sum_{\{s\}} T^{s_1 s_2 s_3 s_4 s_5 s_6} W^{s_1 s_2 s_3 s_4 s_5 s_6} =$$

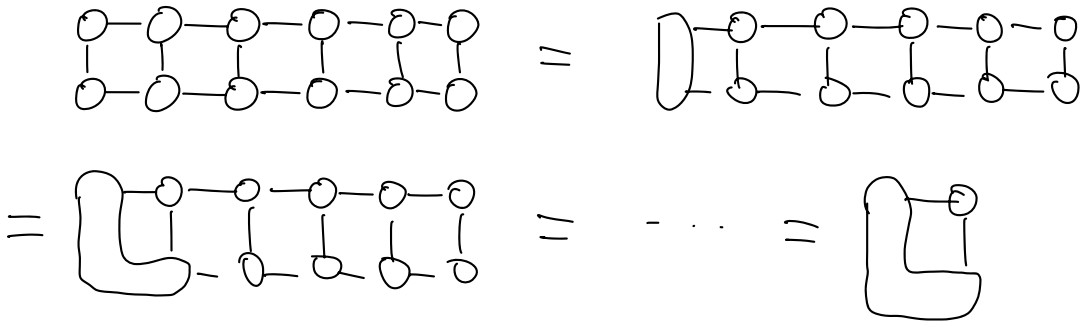



Assume we have $T, W$ as MPS:

$$\begin{cases} T^{s_1 s_2 s_3 s_4 s_5 s_6} = \sum_{\{\alpha\}} A^{s_1}_{\alpha_1} A^{s_2}_{\alpha_1 \alpha_2} \cdots A^{s_6}_{\alpha_5} = \\ W^{s_1 s_2 s_3 s_4 s_5 s_6} = \sum_{\{\beta\}} B^{s_1}_{\beta_1} B^{s_2}_{\beta_1 \beta_2} \cdots B^{s_6}_{\beta_5} = \end{cases}$$

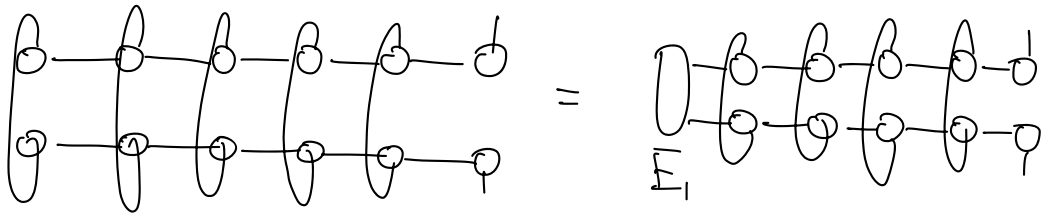


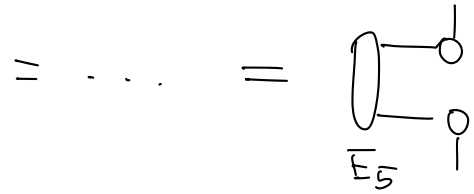



## ③ Compression / Rounding

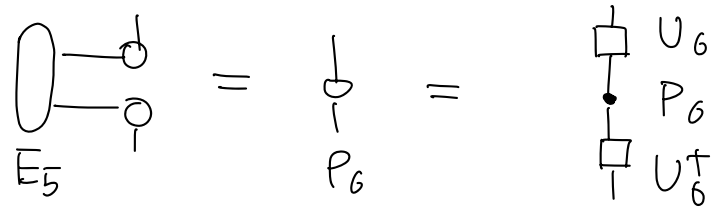
The compression process begins with contracting two copies of the input MPS over all of the external indices, except the last one:




Having computed all of the $E_j$ tensors, it is now possible to form the object $\rho_6$ below, which in physics is called a reduced density matrix.

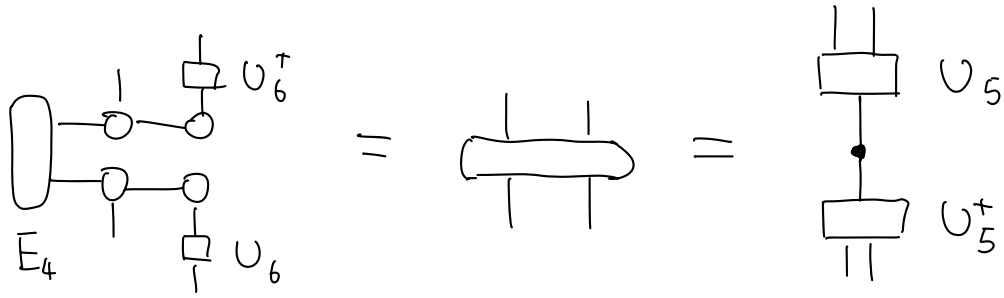
but more simply is the square of the tensor represented by the network, summed over all but the last index:



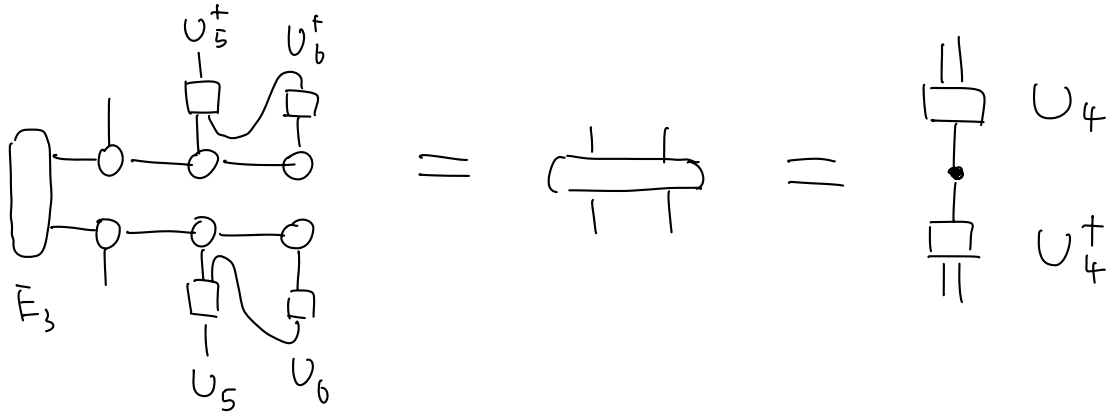

$\rho_6$ is Hermitian, hence can be diagonalized by a unitary $U_6$.

Crucially, at this step one only keeps the $m$ largest eigenvalues of $\rho_6$, discarding the rest and truncating the corresponding columns of $U_6$.

To obtain the next tensor, one next forms the following "density matrix" and diagonalizes it (with truncation):



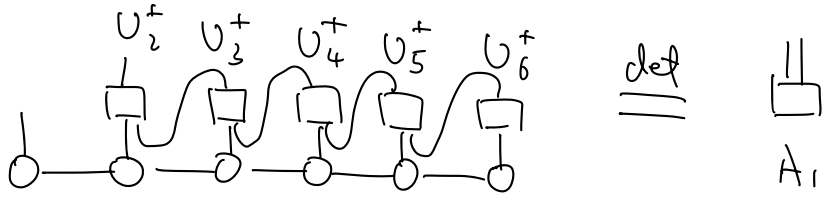

Having obtained $U_5$, one next computes the following density matrix, again using all previous $U$ tensors to rotate and compress the space spanned by all previous external indices:



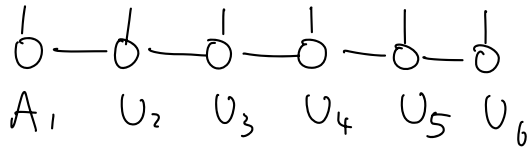

$\rightsquigarrow$ Repeat.

Once all of the $U$ tensors are obtained by repeating the steps above, the last step of the algorithm is to obtain the first tensor of the new

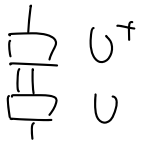
MPS by the following contraction :

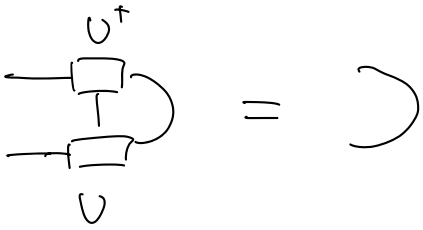



Assembling all of the pieces — the first $A_1$ and $U_i$ $\Longrightarrow$ the final compressed version of the original MPS is




- <u>Gauge and Canonical Forms</u>

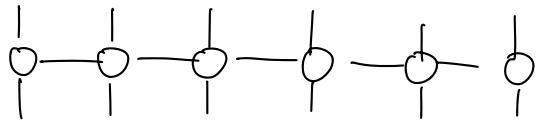
Note that $U^\dagger U = \mathbb{1}$ ,  $= \mathbb{1}$ , or




# 3. Matrix Product Operator (MPO)

An MPO is a factorization of a tensor with N covariant and N contravariant indices into a contracted product of smaller tensors.

$$M^{s_1 s_2 s_3 s_4 s_5 s_6}_{s_1' s_2' s_3' s_4' s_5' s_6'} = \sum_{\{\alpha\}} A^{s_1 \alpha_1}_{s_1'} A^{s_1 \alpha_2}_{\alpha_1 s_2} A^{s_3 \alpha_3}_{\alpha_2 s_3'} A^{s_4 \alpha_4}_{\alpha_3 s_4'} A^{s_5 \alpha_5}_{\alpha_4 s_5'} A^{s_6}_{\alpha_5 s_6'}$$

$=$ 

- <u>Compressing Sums of Local terms</u>

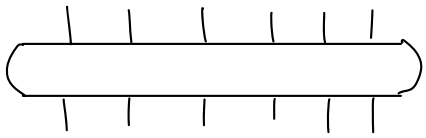
$$A^{s_1'}_{s_1} \otimes \mathbb{1}^{s_2'}_{s_2} \otimes \cdots + \mathbb{1}^{s_1'}_{s_1} \otimes A^{s_2'}_{s_2} \otimes \cdots$$

# 4. Density Matrix Renormalization Group Algorithm (DMRG)

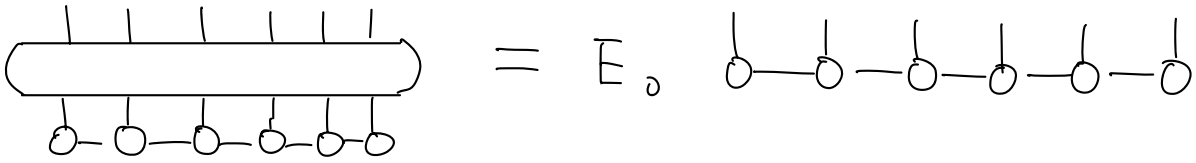
DMRG is a algorithm for optimizing a MPS, s.t. after optimization the MPS is approximately the dominant eigenvectors of a large matrix $H$.

- <u>Statement of the Problem</u>

Consider a Hermitian matrix $H$ acting in vector space that is the tensor product of $N$ smaller spaces, each of dimension $d$:

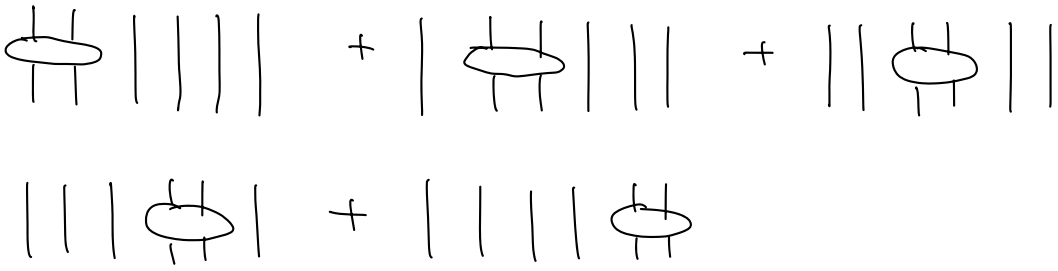
$H =$ 

The DMRG algorithm seeks the dominant eigenvector of $H$ in the form of an MPS

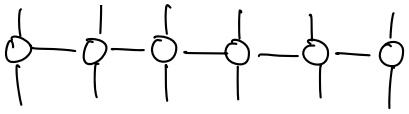



Here $E_0$ ($\leq E_1 \leq E_2 \cdots$) is the minimum eigenvalue of $H$.

For DMRG to be efficient, $H$ must have certain properties. e.g. $H$ could be a sum of local terms:

$H =$ 

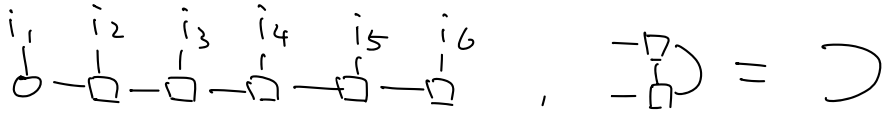
or more generally, could be given as an MPO

$H =$ 

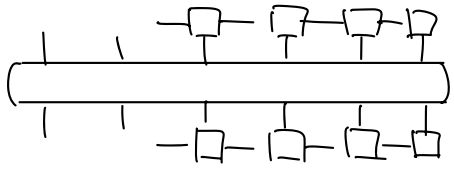
- Steps of DMRG algorithm

(0.) Setup

Initial MPS $\longrightarrow$ orthogonal via a gauge transformation



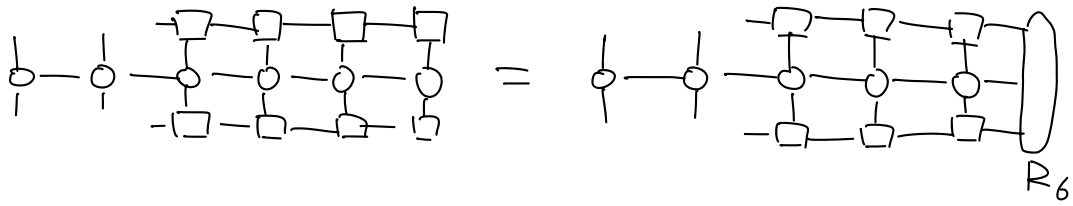

$\Longrightarrow$



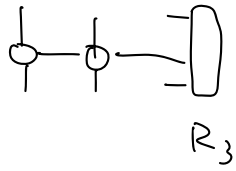
a change of basis from $i_3, i_4, i_5, i_6$ to $\alpha_2$, respectively

Take $H$ to be in MPO form, define $R_j$ tensor:

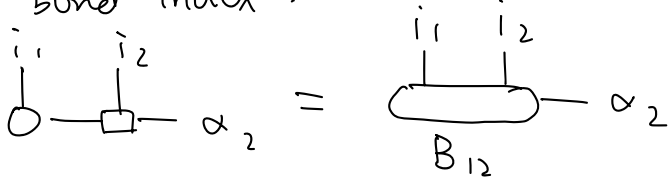
$H =$




$= \cdots =$

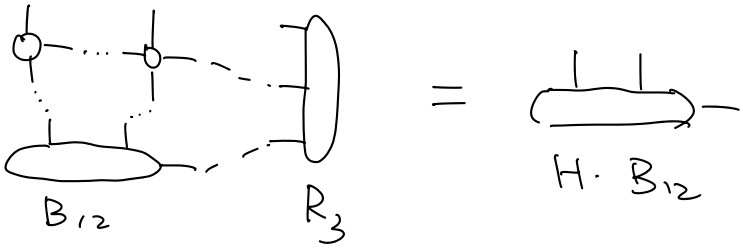



Next, combine the first two MPS tensors by contracting over their shared bond index:




(1a) Optimization of First Bond

Multiplication of $B_{12}$ by the transformed matrix $H$. Using the projected form of $H$ in terms of $R_j$:



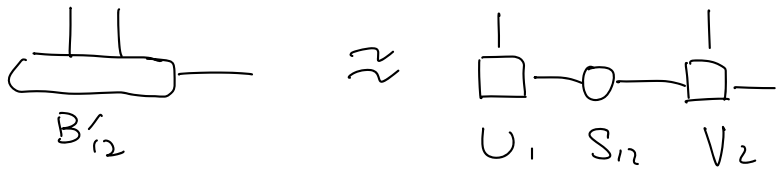

Repeat, until some convergence criterion is met.

$\Longrightarrow B'_{12}$

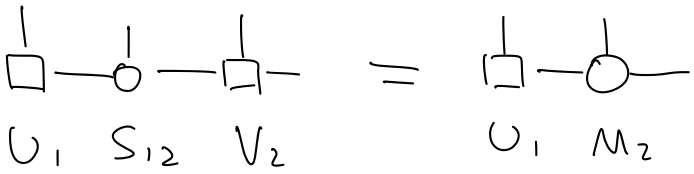
## (1b) Adaptive Restoration of MPS Form

Having obtained an improved bond tensor $B'_{12}$, it is necessary to restore the MPS form of the candidate eigenvector before proceeding to optimize the next bond.

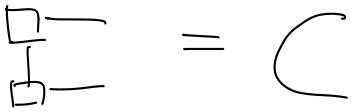
Simplest way: SVD of $B'_{12}$:



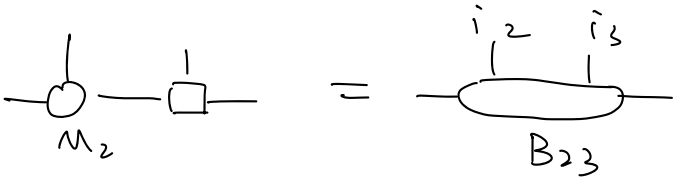

Crucially, one keeps only the first $m$ columns of $U_1$ and rows of $V_2$ corresponding to the $m$ largest sigular values of $B'_{12}$.



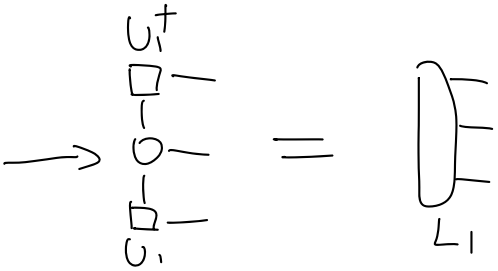

Note that the tensor carrying $i_1$ will automatically obey the left orthogonality property since it is unitary guaranteed by SVD



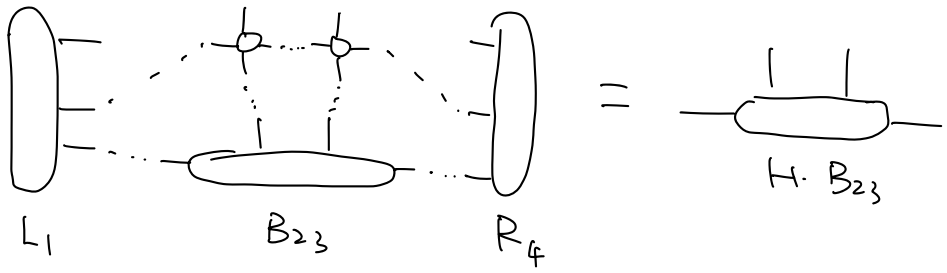
## (2.) Optimization and Adaptation of Second Bond



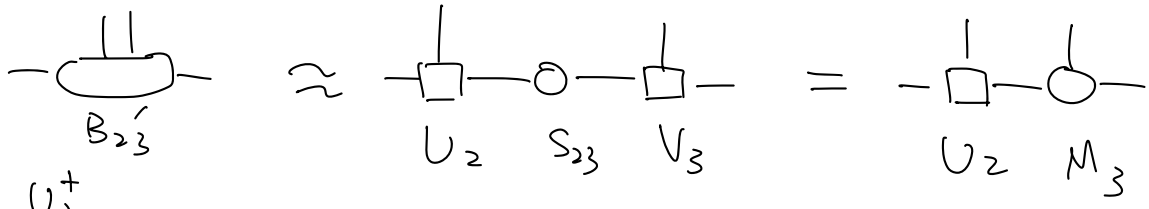

$U_1$ is used to update the projection of $H$ into the basis in which $B_{23}$ is defined.

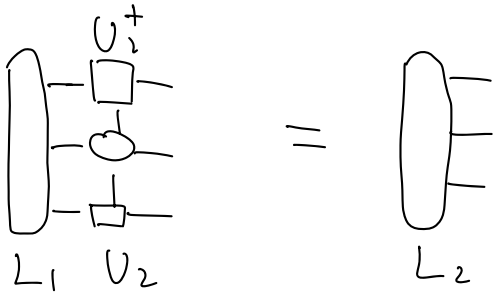



Then $B_{23}$ is optimized via an interative method. with key step being the multiplication of $H$ and $B_{23}$:




After obtaining the improved $B'_{23}$, one again factorizes it to restore MPS form and adapt the bond dimension:




## ③ Remaining Steps and DMRG Sweeping Algorithm

$\longrightarrow$ half sweep
$\longleftarrow$ half sweep
$\Big\} \Longrightarrow$ 1 full sweep